

improved version of Theorem 31.39. Furthermore, there do exist integers  $n$  for which the number of nonwitnesses is  $(n - 1)/4$ .

## Exercises

### 31.8-1

Prove that if an odd integer  $n > 1$  is not a prime or a prime power, then there exists a nontrivial square root of 1, modulo  $n$ .

### ★ 31.8-2

It is possible to strengthen Euler's theorem (Theorem 31.30) slightly to the form

$$a^{\lambda(n)} \equiv 1 \pmod{n} \text{ for all } a \in \mathbb{Z}_n^*,$$

where  $n = p_1^{e_1} \cdots p_r^{e_r}$  and  $\lambda(n)$  is defined by

$$\lambda(n) = \text{lcm}(\phi(p_1^{e_1}), \dots, \phi(p_r^{e_r})).$$

Prove that  $\lambda(n) \mid \phi(n)$ . A composite number  $n$  is a Carmichael number if  $\lambda(n) \mid n - 1$ . The smallest Carmichael number is  $561 = 3 \cdot 11 \cdot 17$ , for which  $\lambda(n) = \text{lcm}(2, 10, 16) = 80$ , which divides 560. Prove that Carmichael numbers must be both “square-free” (not divisible by the square of any prime) and the product of at least three primes. (For this reason, they are not common.)

### 31.8-3

Prove that if  $x$  is a nontrivial square root of 1, modulo  $n$ , then  $\gcd(x - 1, n)$  and  $\gcd(x + 1, n)$  are both nontrivial divisors of  $n$ .

---

## Problems

### 31-1 Binary gcd algorithm

Most computers can perform the operations of subtraction, testing the parity (odd or even) of a binary integer, and halving more quickly than computing remainders. This problem investigates the *binary gcd algorithm*, which avoids the remainder computations used in Euclid's algorithm.

- a.** Prove that if  $a$  and  $b$  are both even, then  $\gcd(a, b) = 2 \cdot \gcd(a/2, b/2)$ .
- b.** Prove that if  $a$  is odd and  $b$  is even, then  $\gcd(a, b) = \gcd(a, b/2)$ .
- c.** Prove that if  $a$  and  $b$  are both odd, then  $\gcd(a, b) = \gcd((a - b)/2, b)$ .
- d.** Design an efficient binary gcd algorithm for input integers  $a$  and  $b$ , where  $a \geq b$ , that runs in  $O(\lg a)$  time. Assume that each subtraction, parity test, and halving takes unit time.

### 31-2 *Analysis of bit operations in Euclid's algorithm*

- a.** Consider the ordinary “paper and pencil” algorithm for long division: dividing  $a$  by  $b$ , which yields a quotient  $q$  and remainder  $r$ . Show that this method requires  $O((1 + \lg q)\lg b)$  bit operations.
- b.** Define  $\mu(a, b) = (1 + \lg a)(1 + \lg b)$ . Show that the number of bit operations performed by EUCLID in reducing the problem of computing  $\gcd(a, b)$  to that of computing  $\gcd(b, a \bmod b)$  is at most  $c(\mu(a, b) - \mu(b, a \bmod b))$  for some sufficiently large constant  $c > 0$ .
- c.** Show that  $\text{EUCLID}(a, b)$  requires  $O(\mu(a, b))$  bit operations in general and  $O(\beta^2)$  bit operations when applied to two  $\beta$ -bit inputs.

### 31-3 *Three algorithms for Fibonacci numbers*

This problem compares the efficiency of three methods for computing the  $n$ th Fibonacci number  $F_n$ , given  $n$ . Assume that the cost of adding, subtracting, or multiplying two numbers is  $O(1)$ , independent of the size of the numbers.

- a.** Show that the running time of the straightforward recursive method for computing  $F_n$  based on recurrence (3.31) on page 69 is exponential in  $n$ . (See, for example, the FIB procedure on page 751.)
- b.** Show how to compute  $F_n$  in  $O(n)$  time using memoization.
- c.** Show how to compute  $F_n$  in  $O(\lg n)$  time using only integer addition and multiplication. (*Hint:* Consider the matrix  $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$  and its powers.)

- d.* Assume now that adding two  $\beta$ -bit numbers takes  $\Theta(\beta)$  time and that multiplying two  $\beta$ -bit numbers takes  $\Theta(\beta^2)$  time. What is the running time of these three methods under this more reasonable cost measure for the elementary arithmetic operations?

### 31-4 Quadratic residues

Let  $p$  be an odd prime. A number  $a \in \mathbb{Z}_p^*$  is a *quadratic residue* modulo  $p$ , if the equation  $x^2 = a \pmod{p}$  has a solution for the unknown  $x$ .

- a.* Show that there are exactly  $(p-1)/2$  quadratic residues, modulo  $p$ .
- b.* If  $p$  is prime, we define the *Legendre symbol*  $\left(\frac{a}{p}\right)$ , for  $a \in \mathbb{Z}_p^*$ , to be 1 if  $a$  is a quadratic residue, modulo  $p$ , and  $-1$  otherwise. Prove that if  $a \in \mathbb{Z}_p^*$ , then

$$\left(\frac{a}{p}\right) = a^{(p-1)/2} \pmod{p}.$$

Give an efficient algorithm that determines whether a given number  $a$  is a quadratic residue, modulo  $p$ . Analyze the efficiency of your algorithm.

- c.* Prove that if  $p$  is a prime of the form  $4k+3$  and  $a$  is a quadratic residue in  $\mathbb{Z}_p^*$ , then  $a^{k+1} \pmod{p}$  is a square root of  $a$ , modulo  $p$ . How much time is required to find the square root of a quadratic residue  $a$ , modulo  $p$ ?
- d.* Describe an efficient randomized algorithm for finding a nonquadratic residue, modulo an arbitrary prime  $p$ , that is, a member of  $\mathbb{Z}_p^*$  that is not a quadratic residue. How many arithmetic operations does your algorithm require on average?

---

## Chapter notes

Knuth [260] contains a good discussion of algorithms for finding the greatest common divisor, as well as other basic number-theoretic algorithms. Dixon [121] gives an overview of factorization and primality

testing. Bach [33], Riesel [378], and Bach and Shallit [34] provide overviews of the basics of computational number theory; Shoup [411] provides a more recent survey. The conference proceedings edited by Pomerance [362] contains several excellent survey articles.

Knuth [260] discusses the origin of Euclid's algorithm. It appears in Book 7, Propositions 1 and 2, of the Greek mathematician Euclid's *Elements*, which was written around 300 B.C.E. Euclid's description may have been derived from an algorithm due to Eudoxus around 375 B.C.E. Euclid's algorithm may hold the honor of being the oldest nontrivial algorithm, rivaled only by an algorithm for multiplication known to the ancient Egyptians. Shallit [407] chronicles the history of the analysis of Euclid's algorithm.

Knuth attributes a special case of the Chinese remainder theorem (Theorem 31.27) to the Chinese mathematician Sun-Tsū, who lived sometime between 200 B.C.E. and 200 C.E.—the date is quite uncertain. The same special case was given by the Greek mathematician Nichomachus around 100 C.E. It was generalized by Qin Jiushao in 1247. The Chinese remainder theorem was finally stated and proved in its full generality by L. Euler in 1734.

The randomized primality-testing algorithm presented here is due to Miller [327] and Rabin [373] and is the fastest randomized primality-testing algorithm known, to within constant factors. The proof of Theorem 31.40 is a slight adaptation of one suggested by Bach [32]. A proof of a stronger result for MILLER-RABIN was given by Monier [332, 333]. For many years primality-testing was the classic example of a problem where randomization appeared to be necessary to obtain an efficient (polynomial-time) algorithm. In 2002, however, Agrawal, Kayal, and Saxena [4] surprised everyone with their deterministic polynomial-time primality-testing algorithm. Until then, the fastest deterministic primality testing algorithm known, due to Cohen and Lenstra [97], ran in  $(1g\ n)^{O(1g\ 1g\ 1g\ n)}$  time on input  $n$ , which is just slightly superpolynomial. Nonetheless, for practical purposes, randomized primality-testing algorithms remain more efficient and are generally preferred.

Beauchemin, Brassard, Crépeau, Goutier, and Pomerance [40] nicely discuss the problem of finding large “random” primes.

The concept of a public-key cryptosystem is due to Diffie and Hellman [115]. The RSA cryptosystem was proposed in 1977 by Rivest, Shamir, and Adleman [380]. Since then, the field of cryptography has blossomed. Our understanding of the RSA cryptosystem has deepened, and modern implementations use significant refinements of the basic techniques presented here. In addition, many new techniques have been developed for proving cryptosystems to be secure. For example, Goldwasser and Micali [190] show that randomization can be an effective tool in the design of secure public-key encryption schemes. For signature schemes, Goldwasser, Micali, and Rivest [191] present a digital-signature scheme for which every conceivable type of forgery is provably as difficult as factoring. Katz and Lindell [253] provide an overview of modern cryptography.

The best algorithms for factoring large numbers have a running time that grows roughly exponentially with the cube root of the length of the number  $n$  to be factored. The general number-field sieve factoring algorithm (as developed by Buhler, Lenstra, and Pomerance [77] as an extension of the ideas in the number-field sieve factoring algorithm by Pollard [360] and Lenstra et al. [295] and refined by Coppersmith [102] and others) is perhaps the most efficient such algorithm in general for large inputs. Although it is difficult to give a rigorous analysis of this algorithm, under reasonable assumptions we can derive a running-time estimate of  $L(1/3, n)^{1.902+o(1)}$ , where  $L(\alpha, n) = e^{(\ln n)^\alpha (\ln \ln n)^{1-\alpha}}$ .

The elliptic-curve method due to Lenstra [296] may be more effective for some inputs than the number-field sieve method, since it can find a small prime factor  $p$  quite quickly. With this method, the time to find  $p$  is estimated to be  $L(1/2, p)^{\sqrt{2}+o(1)}$ .